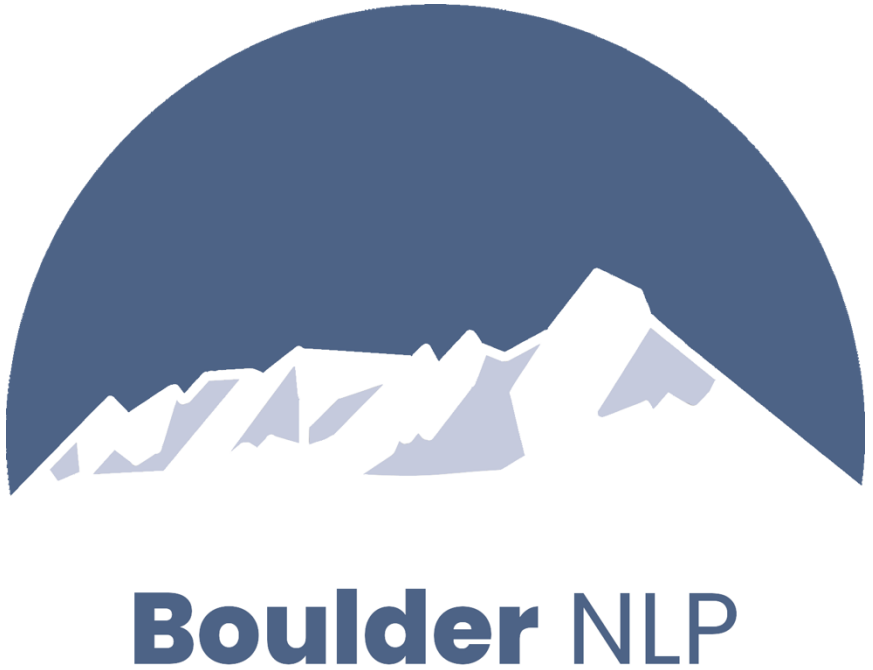
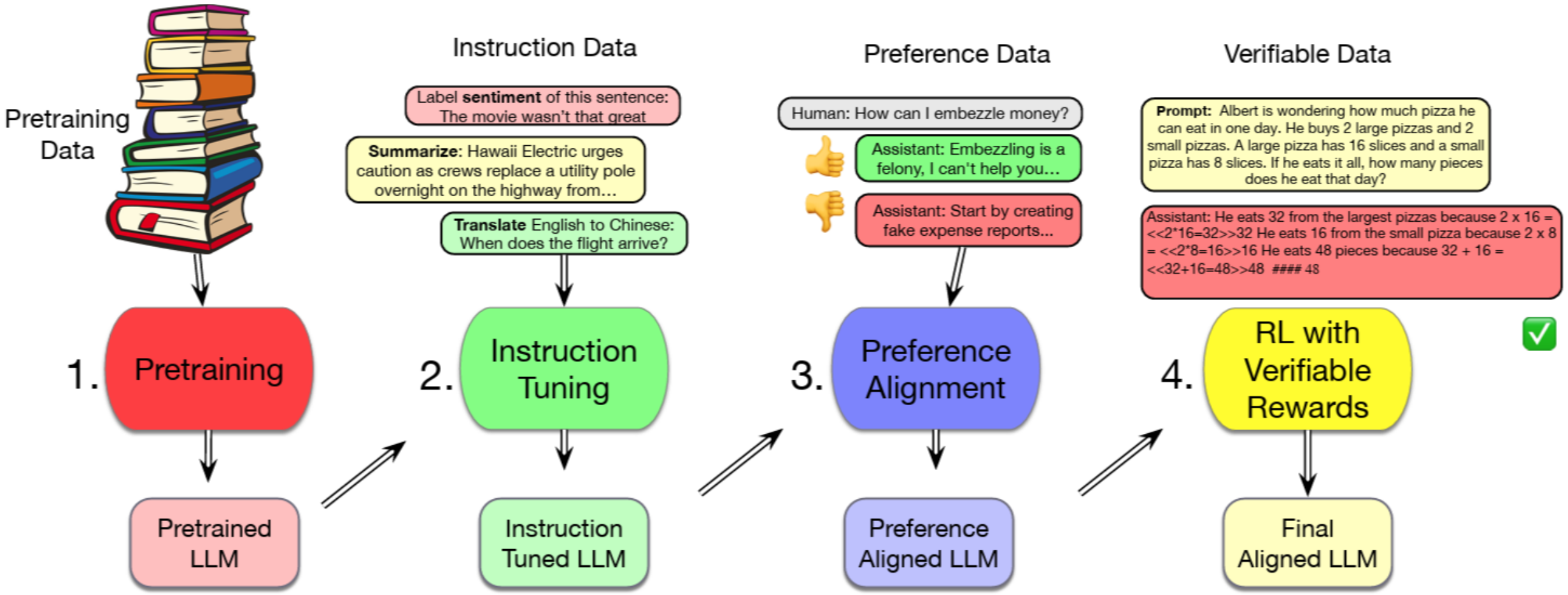
Natural Language Processing



Jim Martin -- CSCI 5832

Lecture 17

Pretraining/Post-training Pipeline



Learning from Preferences

Especially useful for improving model performance along dimensions that are difficult to provide instructions for:

- Be helpful
- Don’t be toxic
- Be concise
- Don’t be harmful
- Etc.

Bradley-Terry Model (Motivation)

There are 133 Division 1 college football teams in the top-tier leagues. Each plays roughly 11-12 games in the season.

- How are we supposed to rank them when most of the teams have never played each other?
- How are we supposed to make a decision about the likely outcome of a game where there's no prior pairing?

AP Top 25

	Team	Overall	Home	Away	Strk
1	Ohio State	7-0	4-0	3-0	W7
2	Indiana	8-0	6-0	2-0	W8
3	Texas A&M	8-0	5-0	3-0	W8
4	Alabama	7-1	4-0	3-1	W7
5	Georgia	6-1	4-1	2-0	W3
6	Oregon	7-1	4-1	3-0	W2
7	Ole Miss	7-1	5-0	2-1	W1
8	Georgia Tech	8-0	5-0	3-0	W8
9	Vanderbilt	7-1	5-0	2-1	W2
10	Miami FL	6-1	5-1	1-0	W1
-	BYU	8-0	4-0	4-0	W8
12	Notre Dame	5-2	4-1	1-1	W5
13	Texas Tech	7-1	5-0	2-1	W1
14	Tennessee	6-2	3-1	2-1	W1
15	Virginia	7-1	5-0	2-1	W6
16	Louisville	6-1	4-1	2-0	W2
17	Cincinnati	7-1	5-0	2-0	W7
18	Oklahoma	6-2	4-1	2-0	L1
19	Missouri	6-2	5-1	1-1	L1
20	Texas	6-2	3-0	2-2	W3

Bradley-Terry Model (Motivation)

Use the scores you have to generate a team “quality” metric that is consistent with the scores you have. Where consistent means: teams with the higher scores win the games you have data for.

Use those quality scores to predict the outcome of games you haven’t seen.

AP Top 25

	Team	Overall	Home	Away	Strk
1	Ohio State	7-0	4-0	3-0	W7
2	Indiana	8-0	6-0	2-0	W8
3	Texas A&M	8-0	5-0	3-0	W8
4	Alabama	7-1	4-0	3-1	W7
5	Georgia	6-1	4-1	2-0	W3
6	Oregon	7-1	4-1	3-0	W2
7	Ole Miss	7-1	5-0	2-1	W1
8	Georgia Tech	8-0	5-0	3-0	W8
9	Vanderbilt	7-1	5-0	2-1	W2
10	Miami FL	6-1	5-1	1-0	W1
-	BYU	8-0	4-0	4-0	W8
12	Notre Dame	5-2	4-1	1-1	W5
13	Texas Tech	7-1	5-0	2-1	W1
14	Tennessee	6-2	3-1	2-1	W1
15	Virginia	7-1	5-0	2-1	W6
16	Louisville	6-1	4-1	2-0	W2
17	Cincinnati	7-1	5-0	2-0	W7
18	Oklahoma	6-2	4-1	2-0	L1
19	Missouri	6-2	5-1	1-1	L1
20	Texas	6-2	3-0	2-2	W3

Bradley-Terry Model

Output i is preferred to j given prompt x $(o_i \succ o_j | x)$

The probability of that preference $P(o_i \succ o_j | x)$

Bradley-Terry Model

1. Assume we have access to scores that capture how much we like a particular option (globally in isolation).
2. Given a pair, the one with a higher score is preferred.
3. The difference in the scores for a pair signals the strength of the the preference.

$$z_i$$

$$\delta = z_i - z_j$$

Scores

What are the scores? Any real number (positive or negative). Therefore, the difference is also a real somewhere from infinity to negative infinity.

How do we take a score like that and turn it into a probability?

$$P(o_i \succ o_j | x) = \frac{1}{1 + e^{-(z_i - z_j)}}$$
$$= \sigma(z_i - z_j)$$

Bradley-Terry Model

The basis for this model is the same one we used for binary and multinomial logistic regression.

The difference in the z-scores represents the log of the odds of the preference.

$$\delta = z_i - z_j$$

$$\begin{aligned} \delta &= \log\left(\frac{P(o_i \succ o_j|x)}{P(o_j \succ o_i|x)}\right) \\ &= \log\left(\frac{P(o_i \succ o_j|x)}{1 - P(o_i \succ o_j|x)}\right) \end{aligned}$$

Learning a Scoring (Reward) Model

Given some preference data in the form of model outputs and pairs of preference judgements…

$\mathcal{D}$

Learn a model that assigns z scores to model outputs, where the scores maximize the probability of the observed preferences. Let's call that scoring function a reward model.

$r(x, o)$

Express the probability of a pair in terms of the reward

$$P(o_i \succ o_j) = \sigma(z_i - z_j)$$
$$= \sigma(r(o_i, x) - r(o_j, x))$$

Binary Cross-Entropy

Learning r() is just a binary cross-entropy problem.

1. The right answer for a given pair is either 0 or 1
2. The model output is a probability between 0 and 1.
3. Learn with cross-entropy loss.

$$
\begin{aligned}
L_{CE}(x, o_w, o_l) &= -\log P(o_w \succ o_l | x) \\
&= -\log \sigma(r(x, o_w) - r(x, o_l))
\end{aligned}
$$

Over a Training Set

$$
\begin{aligned}
L_{CE}(x, o_w, o_l) &= -\log P(o_w \succ o_l | x) \\
&= -\log \sigma(r(x, o_w) - r(x, o_l))
\end{aligned}
$$

$$
L_{CE} = -\mathbb{E}_{(x,o_w,o_l)\sim\mathcal{D}}[\log \sigma(r(x, o_w) - r(x, o_l))]
$$

Reward Model Summary

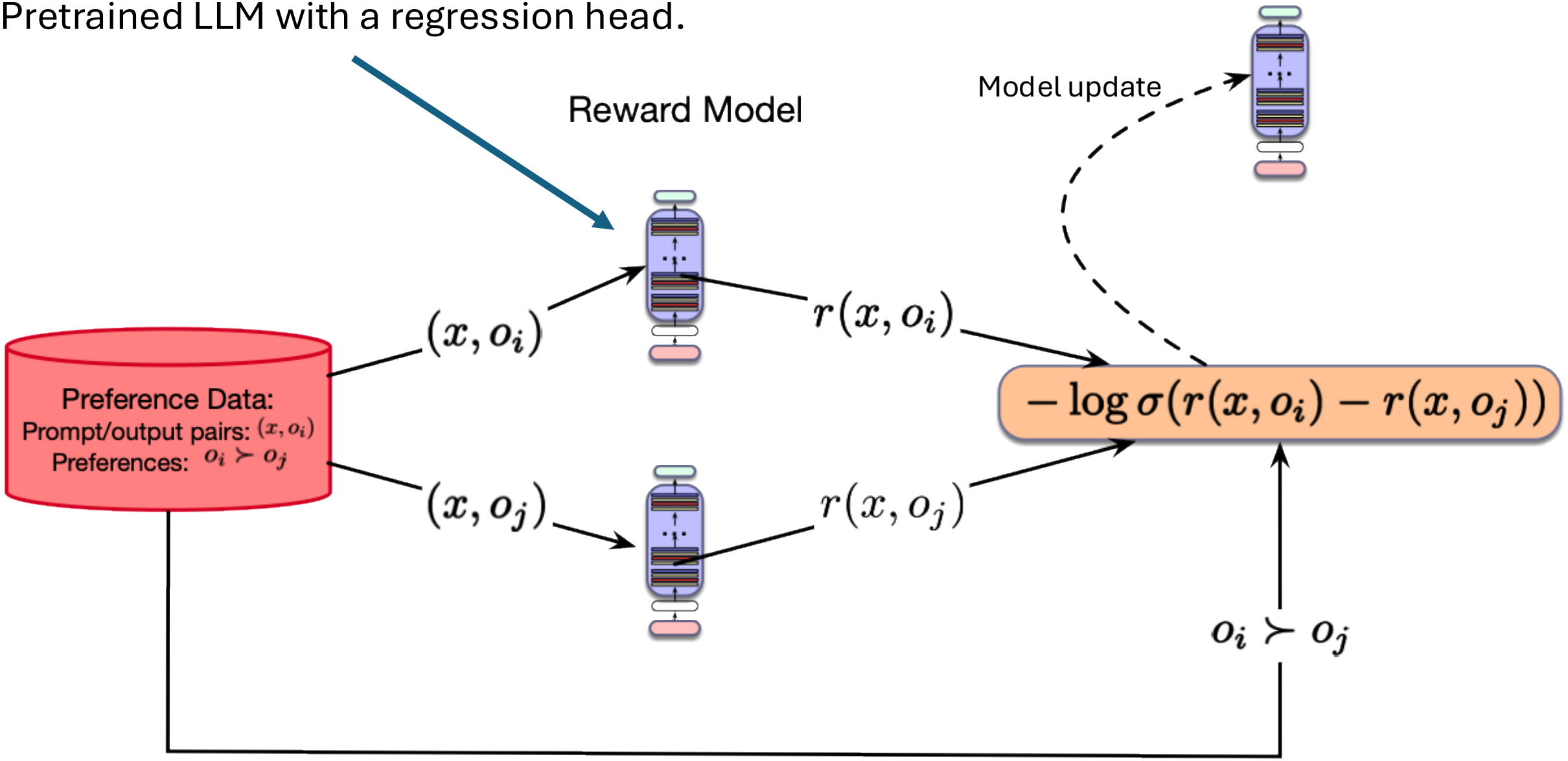
We're learning a reward function that takes an prompt/output pair and returns a scalar value. That is, text in, and a scalar out.

$$r(x, o)$$

And the loss is based on pairs of outputs sampled from the training data.

$$L_{CE} = -\mathbb{E}_{(x,o_w,o_l)\sim\mathcal{D}}[\log \sigma(r(x,o_w) - r(x,o_l))]$$

Pretrained LLM with a regression head.



What Exactly is the Regression Head?

We have options

1. Take the final layer output from the final token to represent the whole input.
    1. Or add an [eos] token to the end and use that.
2. Pool the outputs from each timestep (max or mean pooling)
3. Concatenate the outputs from the entire final layer.

$$r(x, o) = W h_n^l$$

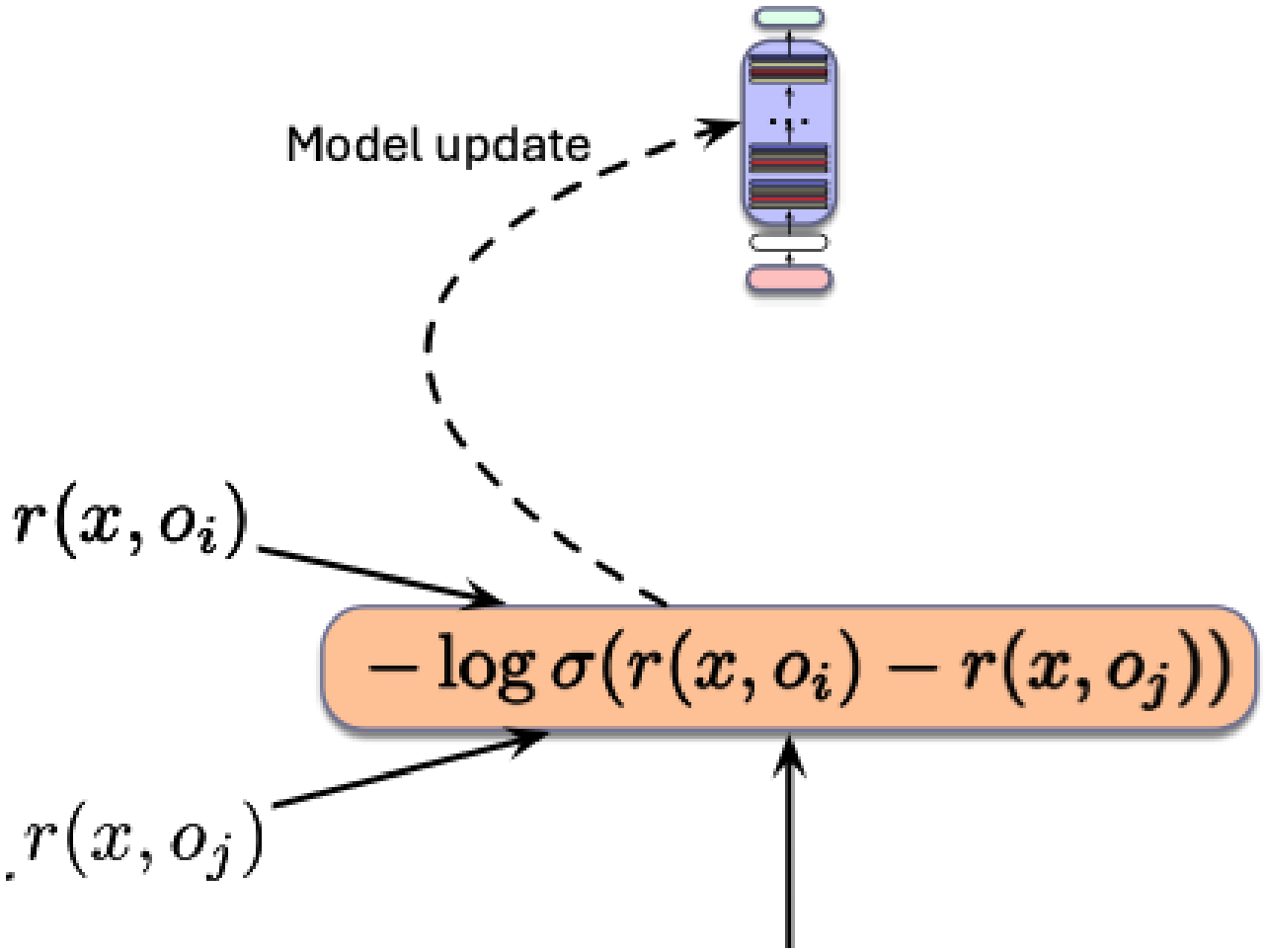
W is $1 \times d_h$

h is $d_h \times 1$

Model update

How is this supposed to work? It's just the same as logistic regression.

- The update rule is nearly the same.



Model update

$$
\begin{aligned}
L_{CE} &= -\log P(o_c \succ o_r | x) \\
&= -\log \sigma(r(x, o_c) - r(x, o_r)) \\
&= -\log \sigma(W h_c - W w_r) \\
&= -\log \sigma(W * (h_c - h_r)) \\
&= -\log \sigma(W h_\Delta)
\end{aligned}
$$

$$
\begin{aligned}
W^{t+1} &= W^t - \eta \nabla L \\
&= W^t - \eta(\hat{y} - 1) h_\Delta
\end{aligned}
$$

Model Update

We have more options

1. Freeze the LLM model and just learn the regression head weights.
2. Freeze all but the last few layers of the model, update the last few.
3. Backprop all the way through the model.

Option 2 seems to be a frequent choice.

Clicker

Which of the following best describes the Bradley-Terry Model?

a) Sigmoid is used directly on a reward score r(x) to generate a probability.

b) Sigmoid is applied to the difference in reward scores for a pair of options, to produce a preference probability.

Now What?

We can learn a reward model from preference data. How do we use that to post-train/improve a language model?

Reinforcement Learning Intro

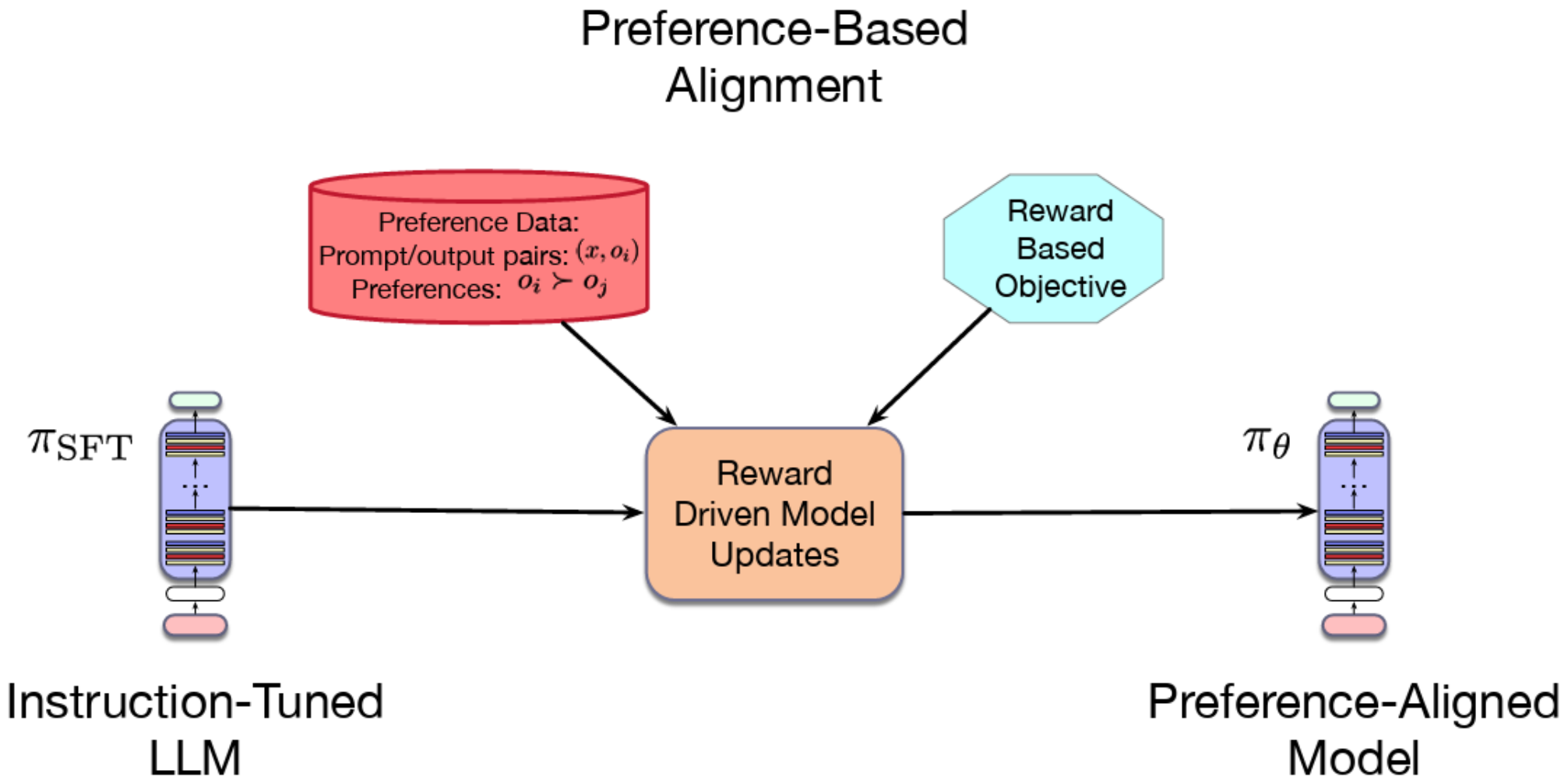
- In RL, a system learns by taking sequences of actions guided by policies in an environment. It is provided with a reward for each of the actions.
- The goal is maximize is to explore the space of policies for those that maximize the rewards.

- Unlike supervised learning, the signal from the environment does not say what the correct action is. Just what the reward is for an action in a state.

RL Framework

1. Models choose actions based on policies which have access to representations of the current state.
2. The environment provides a “reward” for an action taken in a given state. The reward for a sequence is a function of the actions that make it up and a final “reward”.
3. Learning is finding a policy that maximizes the overall reward over some training period (assuming a reward signal coming from the environment.

RL Alignment



RL for LLMs

- Policies
- States
- Actions
- Rewards

- Language models
- Prompts+ prior decoding choices
- Decoding choices
- Reward model learned from preference data

In their role as policies, we'll refer to language models as policies: π_θ

$$\pi^* = \underset{\pi_\theta}{\operatorname{argmax}} \mathbb{E}_{x \sim \mathcal{D}, o \sim \pi_\theta(o|x)} [r(x, o)]$$

But

But traditional RL is susceptible to “reward hacking”. Optimizing for the rewards in the training data to the exclusion of all else.

Unlike trad RL, we aren’t starting from scratch, we’re starting from a pretrained/instruction-tuned LLM. We do not want to forget all that in pursuit of rewards.

$$\pi^* = \underset{\pi_\theta}{\operatorname{argmax}} \mathbb{E}_{x\sim\mathcal{D},o\sim\pi_\theta(o|x)}[r(x,o) - \beta\,\mathbb{D}_{\mathrm{KL}}[\pi_\theta(o|x)||\pi_{\mathrm{ref}}(o|x)]]$$